UNIT-IV

Sub Queries & Transaction control commands

Sub Queries:-

* Introduction:-

In SQL a subquery can be simply defined as a query within another query. In other words we can say that a subquery is a query that is embedded in where clause of another SQL query.

Important rules for subqueries:-

→ You can place the subquery in a number of SQL clauses: WHERE clause, HAVING clause, FROM clause.

Subqueries can be used with SELECT, UPDATE, INSERT, DELETE statements along with expression operator. It could be equality operator or comparison operator such as =, >, <, <=, >= and like operator.

→ A subquery is a query within another query. The outer query is called as main query and inner query is called as subquery.

→ The subquery generally executes first, and its output is used to complete the query condition for the main or outer query.

→ subquery must be enclosed in parentheses.

→ subqueries are on the right side of the comparison operator.

→ ORDER BY command cannot be used in a subquery.

→ GROUP BY command can be used to perform same function as ORDER BY command.

→ Use single-row operators with single row subqueries. Use multiple-row operators with multiple-row subqueries

Syntax:- There is not any general syntax for subqueries. However, subqueries are seen to be used most frequently with SELECT statement.

Syn:-
SELECT column_name
from tablename
where column_name expression operator
(SELECT column_name from table_name where ...);

Ex:-(i) Customers Table:

ID	Name	Age	Address	Salary
1	Ramesh	35	Ahmedabad	2000.00
2	Kailash	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5.	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Ex:- SQL> Select * from customers
where ID IN (Select ID from customers
where salary > 4500);

O/P:-

ID	Name	Age	Address	Salary
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
7	Muffy	24	Indore	10000.00

Subqueries with the Insert Statement:-

Subqueries also can be used with INSERT statements. The INSERT statement uses the data returned from the subquery to insert into another table. The selected data in the subquery can be modified with any of the character, date or number functions.

Insert query example:-

Student1 : Table1

Name	Roll_NO	location	phone_number
Ram	101	chennai	9988773344
Raju	102	coimbatore	9090909090
Ravi	103	salem	8989898989

Student2: Table2

Name	Roll_no	location	phone_number
Raj	111	chennai	8787878787
Sai	112	mumbai	6565656565
Sri	113	coimbatore	78 78 78 78 78

To insert Student2 into Student1 table:

insert into student1 Select * from student2;

O/P:

Name	Roll_no	location	phone_number
Ram	101	chennai	9988773344
Raju	102	coimbatore	9090909090
Ravi	103	salem	8989898989
Raj	111	chennai	8787878787
Sai	112	mumbai	6565656565
Sri	113	coimbatore	7878 78 78 78

(ii) Ex: Sample table:

Database

Name	Roll_no	location	phone_number
Ram	101	chennai	9988 775566
Raj	102	coimbatore	88 77665544
Sasi	103	Madurai	7766 553344
Ravi	104	Salem	8989898989
Sumathi	105	Kanchipuram	8989856868

Student

Name	Roll_no	Section
Ravi	104	A
Sumathi	105	B
Raj	102	A

Sample Query:- To display Name, location, phone_number of the students from Database table whose section is A.

```
Select Name, Location, phone_number from database
 Where Roll_no In
(select Roll_no from Student where section = 'A');
```

Explanation:- First subquery executes "select Roll_no from Student where section = 'A'" returns Roll_no from student table whose section is 'A'. Then outer query executes it and return the Name, location, phone_number from the Database table of the student whose Roll_no is returned from inner table.

Subquery -

O/P:-

Name	Roll_no	location	Phone_number
Ravi	104	salem	8989898989
Raj	102	coimbatore	8877665544

Delete:-

To delete students from student2 table whose roll no is same as that in student1 table and having location as chennai.

```
Delete From Student2
Where Roll_no In ( Select Roll_no from student1
                   where Location = 'chennai' );
```

O/P:- 1 row delete successfully.

Display student2 table:-

Name	Roll_no	location	phone_number
Sai	112	Mumbai	6565656565
Sri	113	coimbatore	7878787878

Update:- To update name of the students to geeks in student2 table whose location is same as Raju, Ravi in student1 table.

```
Update Student2
Set Name = 'geeks'
Where Location In (Select location
                   From Student1
                   where name In ('Raju', 'Ravi'));
```

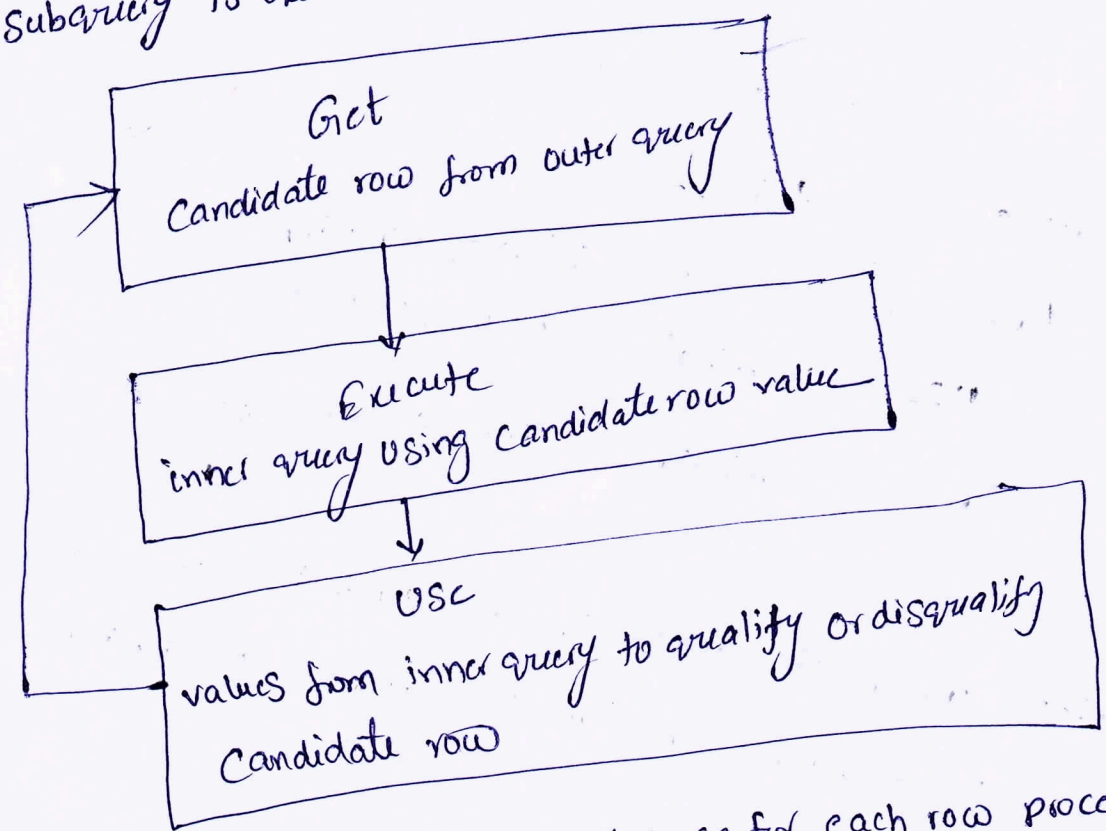
O/P:- 1 row updated successfully.

Display student2 table;

Name	Roll_no	location	Phone_number
Sai	112	mumbai	6565656565
Geeks	113	coimbatore	7878787878

* Correlated Subqueries:- (Synchronized Subquery)

Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.



A correlated subquery is evaluated once for each row processed by the parent statement. The parent statement can be a Select, Update, or Delete statement.

Syn:- Select column1, column2, ...
from table1 outer
where column1 operator (Select column1, column2
from table2
where expr1 = outer.expr2);

ex:-(1) Find all the Employees who earn more than the average salary in their department.

```
Select last_name, salary, department_id
from employees outer
where salary > (select Avg(salary)
                from employees
                where department_id = outer.department_id);
```

Update:-

```
Update table1 alias1
Set column = (Select expression
              from table2 alias2
              where alias1.column = alias2.column);
```

Use a correlated subquery to update rows in one table based on rows from another table.

Delete:-

```
Delete from table1 alias1
where column1 operator
    (Select expression)
    from table2 alias2
    where alias1.column = alias2.column);
```

Use a correlated subquery to delete rows in one table based on the rows from another table.

Ex:- (ii) Find all employees whose salary is above average for their department:

```
Select employee_number, name
from employees emp
where salary > (select Avg(salary)
                from employees
                where department = emp.department);
```

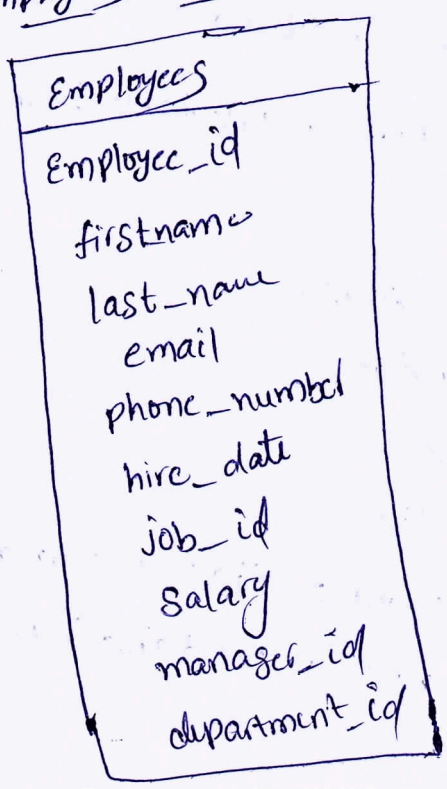
The above query the outer query is

```
Select employee_number, name
from employees emp
where salary > ....
```

The inner query (the correlated subquery) is

```
Select Avg(salary)
from employees
where department = emp.department
```

Ex:(3): Employees table in the sample database:



Query:- 1) To find Employees whose salary is greater than the average salary of all employees:

Command:-

```
Select Employee_id, first_name, last_name,
salary from employees where salary >(select
avg(salary) from employees);
```

Result:

employee_id	first_name	last_name	salary
121	Adam	Fripp	8200.00
103	Alexandar	Hunold	9000.00
109	Daniel	Faviet	9000.00
114	Den	Raphaely	11000.00
204	Hermann	Baer	10000.00
177	Jack	Livingston	8400.00
110	John	chen	8200.00
145	John	Russell	14000.00
176	Jonathon	Taylor	8600.00
146	Karen	partners	13500.00
102	Lex	De haan	17000.00

2) To find all employees whose salary is higher than the average salary of the employees

```
Select Employee_id, first_name, last_name, salary,
department_id from employees e where
salary > ( select Avg(salary) from employees where
department_id = e.department_id)
order by department_id, first_name, last_name;
```

O/P:-

employee_id	first_name	last_name	salary	department_id
201	Michael	Hartstein	13000.00	2
114	Den	Raphaely	11000.00	3
121	Adam	Fripp	8200.00	5
120	Matthew	Weiss	8000.00	5
122	Payam	Kaufling	7900.00	5
123	Shanta	Vollman	6500.00	5
103	Alexander	Hunold	9000.00	6
104	Bruce	Ernst	6000.00	6
145	John	Russell	14000.00	8
146	Karen	Partners	13500.00	8
100	Steven	King	24000.00	9
109	Daniel	Faviet	9000.00	10
108	Nancy	Greenberg	12000.00	10

* Use of Group BY:-

The Group By Statement groups rows that have the same values into summary rows,

(Or)

The Group By statement in SQL is used to arrange identical data into groups with the help of some functions. i.e; if a particular column has same values in different rows then it will arrange these rows in a group.

→ Group By clause is used with the SELECT statement.

→ In the query, Group By clause is placed after the WHERE clause.

→ In the query, Group By clause is placed before ORDER BY clause if used any.

Syn:-

```
SELECT   column_name(s)
From     table_name
Where    condition
GroupBy  column_name(s)
Order BY column_name(s);
```

Sample table:- Employee_table

S.NO.	Name	Salary	Abc
1	Harsh	2000	19
2	Dhanraj	3000	20
3	Ashish	1500	19
4	Harsh	3500	19
5	Ashish	1500	19

→ Group By Single Column:- Group By single column means, to place all the rows with same value of only that particular column in one group.

Query:- Select Name, Sum(salary) From Employee Group By Name;

O/P:

Name	Salary
Ashish	3000
Dhanraj	3000
Harsh	5500

→ Group By multiple Columns:- Group by multiple column is say for example, Group By column1, column2. This means to place all the rows with the same values of both the columns column1 and column2 in one group.

Ex: Student table

Subject	Year	Name
English	1	Harsh
English	1	pratik
English	1	Ramesh
English	2	Ashish
English	2	suresh
mathematics	1	Deepak
mathematics	1	sayan

Query:

```
Select subject, year, count(*)
from student
Group By subject, year;
```

O/P:

Subject	Year	Count
English	1	3
English	2	2
mathematics	1	2

* Having :-

The Having clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

The Having clause enables you to specify conditions that filter which group results appear in the final results.

The where clause places conditions on the selected columns, where as the Having clause places conditions on groups created by the Group By clause.

Syn:- Select column_name(s)
from table_name
Where Condition
Group By column_name(s)
Having Condition
Order By column_name(s);

Ex:- Customers table

ID	Name	Age	Address	Salary
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	delhi	1500.00
3	kaushik	23	kota	2000.00
4	chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	komal	22	MP	4500.00
7	Mufty	24	Indore	10000.00

Query:- SQL> Select ID, Name, Age, Address, Salary
from customers
Group By age
Having Count (age) >=2;

O/P:-

ID	Name	Age	Address	Salary
2	Khilan	25	Delhi	1500.00

* Order BY:-

The ORDER BY statement in SQL is used to sort the fetched data in either ascending or descending according to one or more columns.

→ By default Order By sorts the data in ascending order.

→ We can use the keyword DESC to sort the data in descending order and the keyword ASC to sort in ascending order.

Syn:- Select column$_1$, column$_2$, ...
from table_name
Order By column$_1$, column$_2$, ...
ASC | DESC;

→ Sort according to one column:- To sort in ascending or descending order we can use the keywords ASC or DESC.

Syn:- Select * from table_name Order By column_name
ASC | DESC;

Ex:-

Roll_no	Name	Address	Phone	Age
1	Harsh	Delhi	xxxxxxxxx	18
2	pratik	Bihar	xxxxxxxxx	19
3	Riyanka	siliguri	xxxxxxxxx	20
4	deep	Ramnagar	xxxxxxxxx	18
5	saptarhi	kolkata	xxxxxxxxx	19
6	dhanraj	Bara Bajar	xxxxxxxxx	20
7	Rohit	Baluraghat	xxxxxxxxx	18
8	Niraj	Alipur	xxxxxxxxx	19

→ Atomic :- In a transaction involving two or more pieces of information, either all or none of the information is saved. Basically, a rule of "all or nothing" is observed.

→ Consistent :- The saved data cannot violate the integrity of the database. Interrupted modifications are rolled back to ensure the database is in a state before the change takes place.

→ Isolation :- No other transactions take place and affect the transaction in question. This prevents "mid-air collisions"

→ Durable :- once the transaction is committed, any failure or system restart, returns the data in a correct state. Stated another way, once a transaction is committed, it will remain so, regardless of a subsequent system failure.

```
Select * from Student Order By Roll_no DESC;
```

O/p:-

Roll_no	name	Address	phone
8	Niraj	Alipur	xxxxxxxxxx
7	Rohit	Balurghat	xxxxxxxxxx
6.	Dhanraj	Barabajar	xxxxxxxxxx
5	saptarhi	Kolkata	xxxxxxxxxx
4	Deep	Ramnagar	xxxxxx xxxx
3	Riyanka	siliguri	xxx xxxxxxx
2	Pratik	Bihar	xxxxxxxxxx
1	Harsh	Delhi	xxxxxxxxxx

→ Sort according to multiple columns:-

To sort according to multiple columns, separate the names of columns by (,) operator.

Syn:- Select * from table_name order By column1 ASC|DESC, column2 ASC|DESC;

Roll_no	Name	Address	phone	Age
1	Harsh	Delhi	xxxxxxxxxx	18
2	pratik	Bihar	xxxxxxxxxx	19
3.	Riyanka	siliguri	xxxxxxxxxx	20
4	Deep	Ramnagar	xxxxxxxxxx	18
5	Saptagiri	Kolkata	xxxxxxxxxx	19
6	Dhanraj	Barabajar	xxxxxxxxxx	20.
7	Rohit	Balurghat	xxxxxxxxxx	18
8	Niraj	Alipur	xxxxxxxxxx	19

Ex:-2. Select * From Student Order By Age ASC, Roll_no DESC;

O/P:-

Roll_no	Name	Address	Phone	Age
7	Rohit	Balurghat	xxxxxxxxx	18
4	Deep	Ramnagar	xxxxxxxxxx	18
1	Harsh	Delhi	xxxxxxxxxx	18
8	Niraj	Aliput	xxxxxxxxxx	19
5	Saptarhi	Kolkata	xxxxxxxxxx	19
2	pratik	Bihar	xxxxxxxxxx	19
6	Dhanraj	Barabajar	xxxxxxxxxx	20
3	Riyanka	Siliguri	xxxxxxxxxx	20

* Join and its types:-

Join:- A join clause is used to combine rows from two or more tables, based on a related column between them.

(Or)

The SQL Join clause is used to combine records from two or more tables in a database. A join is a means for combining fields from two tables by using values common to each.

Ex:. Customers table

ID	Name	Age	Address	Salary
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Mufty	24	Indore	10000.00

Orders table

OID	Date	customer_ID	Amount
102	2009-10-08 00:00:00	3	3000
100	2009-10-08 00:00:00	3	1500
101	2009-11-20 00:00:00	2	1560
103	2008-05-20 00:00:00	4	2060

Now, let us join these two tables in our select statement

SQL> Select ID, Name, Age, Amount
from customers, Orders
where customer.ID = Orders.customer_ID;

Result:

ID	Name	Age	Amount
3	kaushik	23	3000
3	kaushik	23	1500
2	Khilan	25	1560
4	Chaitali	25	2060

* Different Types of SQL Joins:-

→ (Inner) Join:- Returns records that have matching values in both tables.

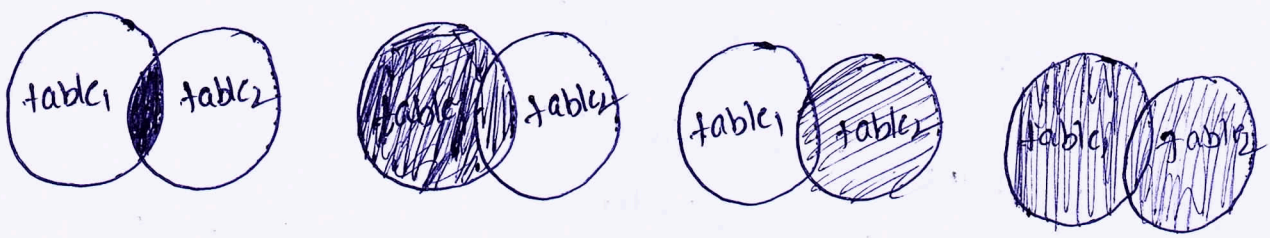
→ Left (outer) Join:- Returns all records from the left table, and the matched records from the right table.

→ Right (outer) Join:- Returns all records from the right table, and the matched records from the left table.

→ Full (outer) Join:- Returns all records when there is a match in either left or right table.

→ Self Join:- A self join is a regular join, but the table is joined with itself.

Inner join | left join | Right join | Full outer join



1) Inner Join:- The inner join keyword selects records that have matching values in both tables.

Syn:- 1) Select column_name(s)
From table1
Inner Join table2
ON table1.column_name = table2.column_name;

Ex:- Student table

Roll_no	Name	Address	Phone	Age
1	Harsh	Delhi	**********	18
2	pratik	Bihar	**********	19
3	priyanka	siliguri	**********	20
4	Deep	Ramnagar	**********	18
5	Saptarhi	Kolkata	**********	19
6.	Dhanraj	Barabajar	**********	20
7	Rohit	Balurghat	**********	18
8.	Niraj	Alipur	**********	19

StudentCourse:-

Course_ID	Roll_no
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

Syntax:- Select table1.column1, table1.column2, table2.column1, ...
From table1
Inner Join table2
ON table1.matching_column = table2.matching col.

Query:- To find the names and age of students enrolled in different courses:

Command:

```
Select StudentCourse.Course_ID, Student.Name,
    Student.Age from Student
Inner Join StudentCourse
ON Student.Roll_no = StudentCourse.Roll_no;
```

O/P:-

Course_ID	Name	Age
1	Harsh	18
2	pratik	19
2	Riyanka	20
3	Deep	18
1	Saptarhi	19

(2) Left Join:- This join returns all the rows of the table on the left side of the join and matching rows for the table on the right side of join. The rows for which there is no matching row on right side, the result-set will contain null. Left join is also known as Left outer join.

Syn:- Select table1.column1, table1.column2, table2.column1, ...
from table1
Left Join table2
ON table1.matching_column = table2.matching_column;

Query:-

```
Select Student.Name, StudentCourse.Course_ID
from student
left Join Studentcourse
ON StudentCourse.Roll_no = Student.Roll_no;
```

O/P:-

Name	Course_ID
Harsh	1
Pratik	2
Riyanka	2
Deep	3
Saptarhi	1
Dhanraj	NULL
Rohit	NULL
Niraj	NULL

(3) Right Join:- Right Join is similar to left join. This join returns all the rows of the table on the right side of the join and matching rows for the table on the left side of join. The rows for which there is no matching row on left side, the result-set will contain null. Right Join is also known as Right outer join.

Syn:-
```
Select table1.column1, table1.column2, table2.col1, ...
from table1
Right Join table2
ON table1.matching_column = table2.matching_column;
```

Query:-

```
Select Student.Name, StudentCourse.Course_ID
from Student
Right Join StudentCourse
ON StudentCourse.Roll_no = Student.Roll_no;
```

O/P:-

Name	Course_ID
Harsh	1
pratik	2
Riyanka	2
Deep	3
saptarhi	1
NULL	4
NULL	5
NULL	4

4) Full Join:- Full Join creates the result-set by combining result of both left Join and Right Join. The result-set will contain all the rows from both the tables. The rows for which there is no matching, the result-set will contain NULL values.

Syn:-

```
Select table1.column1, table1.column2, table2.column1, ...
from table1
full Join table2
ON table1.matching_column = table2.matching_column;
```

Query:

```
Select Student.Name, StudentCourse.Course_ID
from Student
Full Join StudentCourse
ON StudentCourse.Roll_no = Student.Roll_no;
```

O/P:-

Name	Course_ID
Harsh	1
pratik	2
Riyanka	2
Deep	3
saptarhi	1
Dhanraj	NULL
Rohit	NULL
Niraj	NULL
NULL	9
NULL	10
NULL	11

5) Self Join:- A self join is a regular join, but the table is joined with itself.

Syn:- Select column_name(s)
from table1 t1, table1 t2
where condition;

(Or)

As the name signifies, in self join a table is joined to itself. That is, each row of the table is joined with itself and all other rows depending on some conditions. In other words we can say that it is a join between two copies of the same table

Syn:- Select a.column1, b.column2
From table_name a, table_name b
Where some_condition;

Consider two tables:-

Student

Roll_no	Name	Address	Phone	Age
1	Ram	Delhi	xxxxxxxxxx	18
2	Ramesh	Gurgaon	xxxxxxxxxx	19
3	Sujit	Rohtak	xxxxxxxxxx	20
4	Suresh	Delhi	xxxxxxxxxx	18

Student course

Course_ID	Roll_no
1	1
2	2
2	3
3	4

Query:- Select a.Roll_no, b.Name
from Student a, Student b
where a.Roll_no < b.Roll_no;

O/P:-

Roll_no	Name
1	Ramesh
1	Sujit
2	Sujit
1	Suresh
2	Suresh
3	Suresh

6) Cartesian Join:- The cartesian Join is also known as Cross Join. In a cartesian Join there is a Join for each row of one table to every row of another table. This usually happens when the matching column or where condition is not specified.

→ In the absence of a where condition the cartesian Join will behave like a cartesian Product. i.e; the number of rows in the result-set is the product of the number of rows of the two tables.

→ In the presence of where condition this Join will function like a Inner Join.

→ Generally Cross join is similar to an inner Join where the Join-condition will always evaluate to true.

Syn:- Select table1.column1, table1.column2, table2.column1, ...
From table1
Cross Join table2;

In the below query we will select Name and Age from Student table and Course_ID from StudentCourse table. In the output we can see that each row of the table Student is Joined with every row of the table StudentCourse. The total rows in the result-set = 4 * 4 = 16.

Select student.Name, Student.Age, StudentCourse.
Course_ID from student
Cross Join StudentCourse;

O/P:-

Name	Age	course_ID
Ram	18	1
Ram	18	2
Ram	18	2
Ram	18	3
Ramesh	18	1
Ramesh	18	2
Ramesh	18	2
Ramesh	18	3
Sujit	20	1
Sujit	20	2
Sujit	20	2
Sujit	20	3
Suresh	18	1
Suresh	18	2
Suresh	18	2
Suresh	18	3

* **Exists:-** The EXISTS operator is used to test for the existence of any record in a subquery.

The Exists Condition in SQL is used to check whether the result of a correlated nested query is empty (contains no tuples) or not. The result of exists is a boolean value True or False. It can be used in a Select, Update, insert or delete statement.

Syn:-
```
Select column_name(s)
From table_name
Where EXISTS
(Select column_name(s)
from table_name
Where condition);
```

The EXISTS operator returns true if the subquery returns one or more records.

Ex:- Consider the following two relation "customers" and "orders".

customer table.

customer_id	lname	fname	website
401	singh	Dolly	abc.com
402	chauhan	Anuj	def.com
403	kumar	Nitesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

Orders table

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
3	408	2017-01-18
4	404	2017-02-05

(1) Using Exists condition with select statement :-

To fetch the first and last name of the customers who placed atleast one order.

```
Select fname, lname
from customers
Where EXISTS (Select * from Orders
Where customers.customer_id =
Orders.c_id);
```

O/P:-

fname	lname
Shubham	Gupta
Divya	Walecha
Rajiv	Mehta
Anand	Mehra

2) Using NOT with EXISTS:-

Fetch last and first name of the customers who has not placed any order.

```
Select lname, fname
from customer
where NOT EXISTS (select * from orders
                  where customers.customer_id =
                  Orders.c_id);
```

O/P:-

lname	fname
Singh	Dolly
Chauhan	Anuj
Kumar	Niteesh
Jain	Sandeep

3) Using EXISTS condition with Delete statement

Delete the record of all the customer from Order table whose lastname is 'Mehra'.

```
Delete from Orders
where EXISTS( select * from customers
              where customers.customer_id =
              Orders.cid And customers.lname
              = 'Mehra');
```

```
Select * from orders;
```

O/P:-

Order_id	c_id	Order_date
1	407	2017-03-03
2	405	2017-03-05
4	404	2017-02-05

4) Using Exists condition with Update statement

Update the lname 'kumari' of customer in customer table whose customer_id is 401.

```
Update customers
  Set lname = 'kumari'
  where Exists (select * from customers
                where customer_id = 401);
```

```
select * from customers;
```

O/P:-

customer_id	lname	fname	website
401	Kumari	Dolly	abc.com
402	chauhan	Anuj	def.com
403	Kumar	Nitesh	ghi.com
404	Gupta	shubham	jkl.com
405	Walccha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

* Any:- The ANY operator is used with a WHERE or HAVING clause.

-> The ANY operator returns true if any of the subquery values meet the condition.

-> ANY must be preceded by comparison operators.

Syn:- Select column_name(s)
From table_name
Where column_name operator ANY
(Select column_name from table_name WHERE condition);

(or)

Select column_name(s)
From table_name
Where column_name comparison_operator ANY
(Select column_name
from table_name
Where condition(s));

Queries: Ex: Consider the following Products table and OrderDetails table.

Product table

ProductID	ProductName	SupplierID	CategoryID	price
1	Chais	1	1	18
2	Chang	1	1	19
3	Aniseed Syrup	1	2	10
4	Chef Anton's Cajun Seasoning	2	2	22
5	Chef Anton's Gumbo Mix	2	2	21
6	Boysenberry Spread	3	2	25
7	Organic Dried Pears	3	7	30
8	Northwoods Cranberry Sauce	3	2	40
9	Mishi Kobe Niku	4	6	97

Order Details table

OrderDetailsID	OrderID	ProductID	Quantity
1	10248	1	12
2	10248	2	10
3	10248	3	15
4	10249	1	8
5	10249	4	4
6	10249	5	6
7	10250	3	5
8	10250	4	18
9	10251	5	2
10	10251	6	8
11	10252	7	9
12	10252	8	9
13	10250	9	20
14	10249	9	4

Queries:- 1) Find the Distinct categoryID of the products which have any record in OrderDetails table.

```
Select Distinct categoryID
from products
Where productID = ANY (select productID from
                       OrderDetails);
```

O/P:-

categoryID
1
2
7
6

2) Finds any records in the OrderDetails table that Quantity = 9

```
Select ProductName
from products
Where productID = ANY (Select productID
                       from OrderDetails
                       Where Quantity = 9);
```

O/P:-

ProductName
Organic Dried pears.
Northwoods Cranberry sauce

* ALL:- ALL & ANY are logical operators in SQL. They return boolean value as a result.

→ ALL operator is used to select all tupks of SELECT statement It is also used to compare a value to every value in another value set or result from a subquery.

→ The ALL operator returns TRUE iff all of the subqueries values meet the condition. The ALL must be preceded by comparison operators and evaluates true if all of the subqueries values meet the condition.

→ All is used with SELECT, WHERE, HAVING statement.

Syntax:- Select column_name(s)
from table_name
where column_name operator ALL
(Select column_name from table_name where condition);

→ ALL with Select statement:-

Syn:- Select ALL field_name
from table_name
where condition(s);

→ All with WHERE or HAVING Statement:-

Syn:- Select column_name(s)
from table_name
Where column_name comparison_operator ALL
(select column_name from table_name
Where condition(s));

Queries:- 1) Find the name of the all the product

```
Select ALL productName
from products
Where True;
```

O/P:-

Product Name
Chais
Chang
Aniseed Syrup
Chef Anton's Cajun Seasoning
Chef Anton's Gumbo Mix
Boysenberry Spread
Organic Dried Pears
Northwoods Cranberry Sauce
Mishi Kobe Niku

2) Find the name of the product if all the records in the OrderDetails has quantity either equal to 6 or 2.

```
Select productName
from products
where productID = ALL ( select productID
                        from OrderDetails
                        where Quantity = 6 OR Quantity = 2);
```

O/P:-

ProductName
Chef Anton's
Gumbo Mix

3) Find the orderID whose maximum quantity among all product of that orderID is greater than average quantity of all orderID.

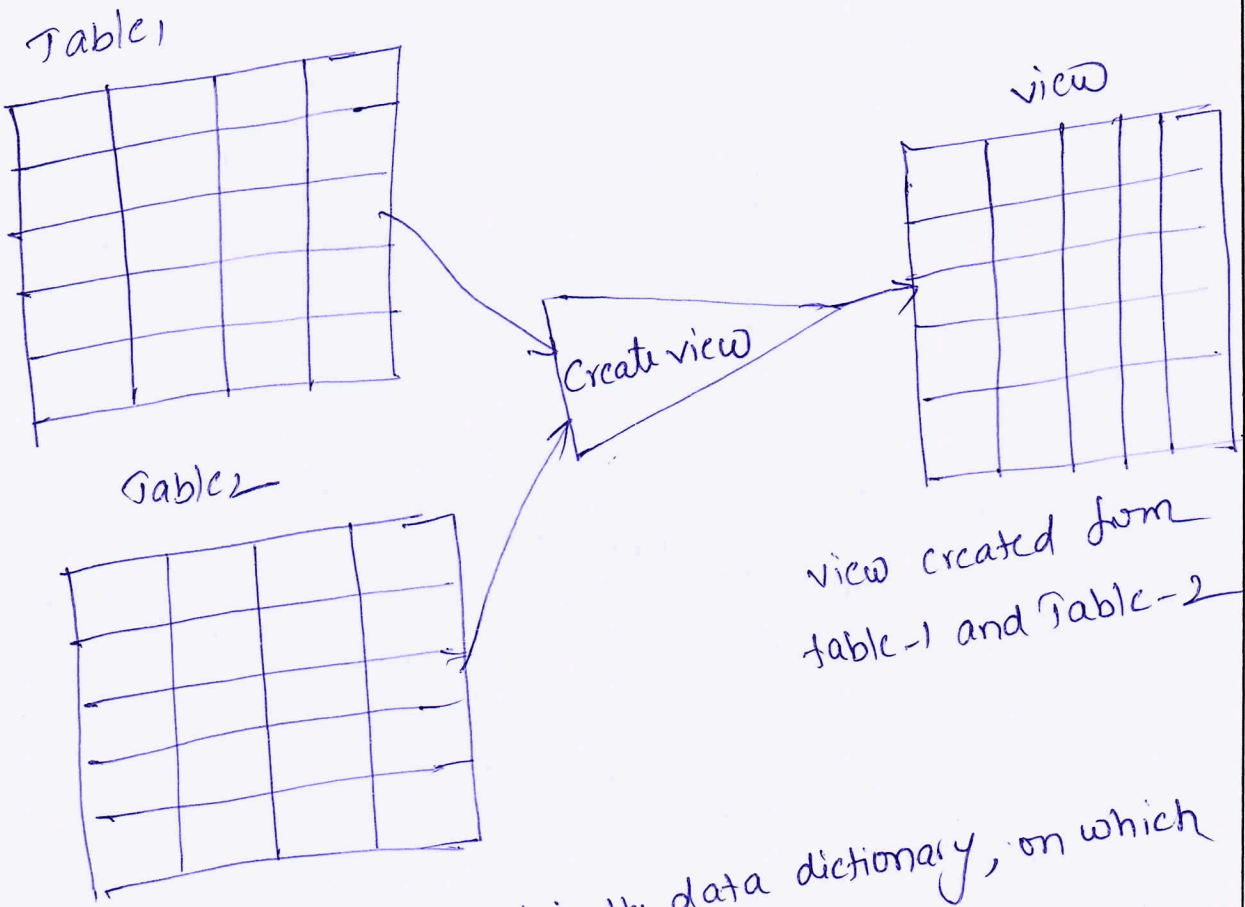
```
Select OrderID
from OrderDetails
GroupBy OrderID
Having max(Quantity) > ALL (select avg(Quantity)
                            from OrderDetails
                            GroupBy OrderID);
```

O/P:-

OrderID
10248
10250

*View & its types!-

View:- Views are a special version of tables in SQL. They provide a virtual table environment for various complex operations. We can select data from multiple tables, or we can select specific data based on ~~criteria~~ certain criteria in views. It does not hold the actual data. It holds only the definition of the view in the data dictionary.

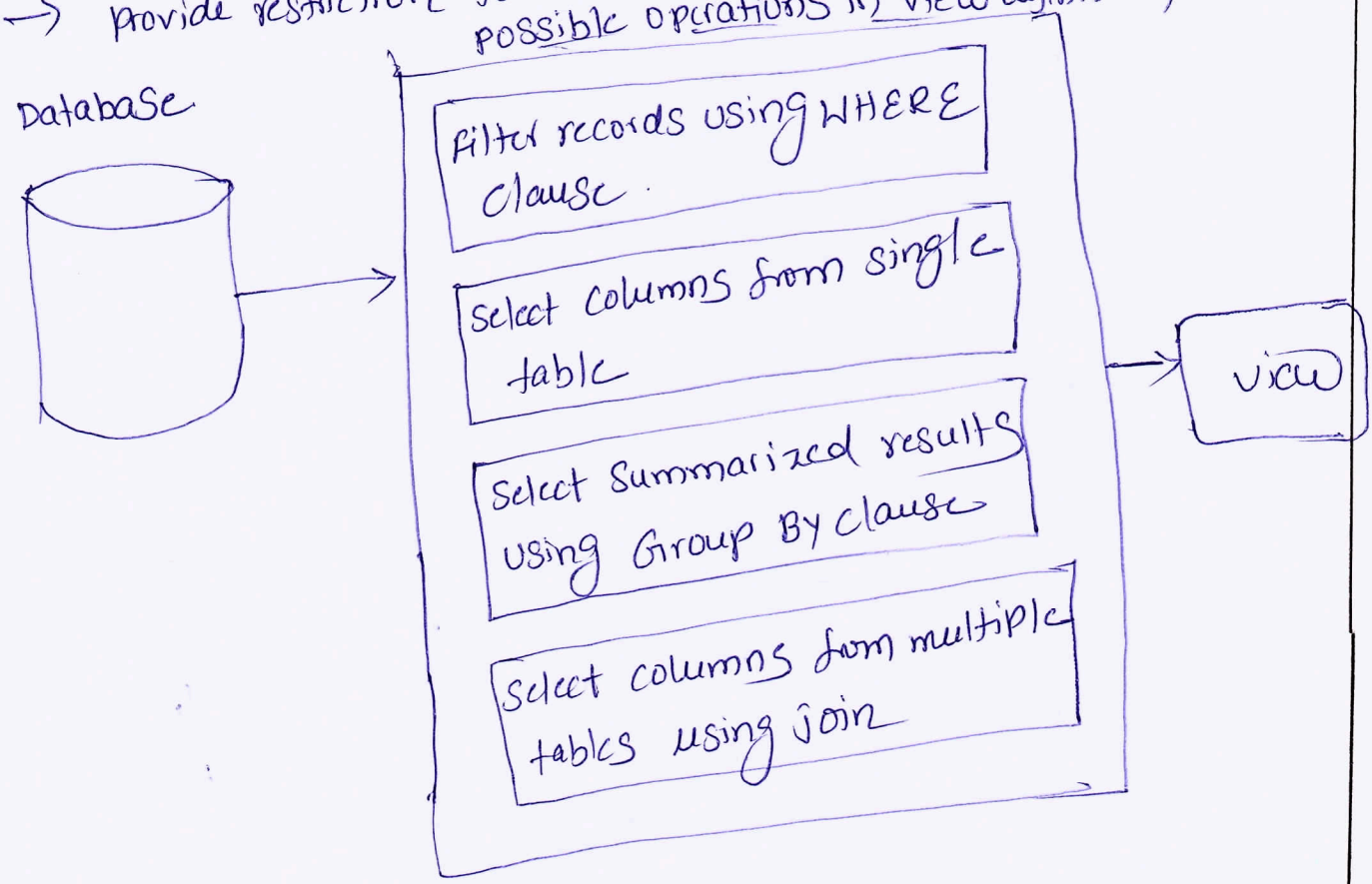


The view is a query stored in the data dictionary, on which the user can query just like they do on tables. It does not use the physical memory, only the query is stored in the data dictionary. It is computed dynamically, whenever the user performs any query on it. Changes made at any point in view are reflected in the actual base table

The view has primarily two purposes:

→ Simplify the complex SQL queries.

→ Provide restriction to users from accessing sensitive data.



→ Create view Statement:-

In SQL, a view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

We can add SQL functions, WHERE, and Join statements to a view and present the data as if the data were coming from one single table.

Syn:- Create view view_name AS
Select column1, column2, ...
From table_name
Where condition;

→ SQL updating a view:-

A view can be updated with the Create or Replace view command.

SQL Create OR Replace view syntax:-

Syn:
Create OR Replace view view_name AS
Select column1, column2, ...
From table_name
Where condition;

→ SQL Dropping a view:-

A view is deleted with the Drop view command.

Syn:- Drop view view_name;

* Types of views:-

1) Simple view:- A view based on only a single table, which doesn't contain Group By clause and any functions.

2) Complex view:- A view based on multiple tables, which contain Group By clause and functions.

3) Inline view:- A view based on a subquery in from clause, that subquery creates a temporary table and simplifies the complex query.

4) Materialized view:- A view that stores the definition as well as data. It creates replicas of data by storing it physically.

1) Simple view Example:-

Employee.

EmployeeID	Ename	DeptID	salary
1001	John	2	4000
1002	Anna	1	3500
1003	James	1	2500
1004	David	2	5000
1005	Mark	2	3000
1006	Steve	3	4500
1007	Alice	3	3500

Create view of grouped records on employee table

```
Create view emp_view AS
Select Employee_ID, Ename
from Employee
Where DeptID = 2;
```

O/P:-

EmployeeID	Ename	DeptID	Salary
1001	John	2	4000
1004	David	2	5000
1005	Mark	2	3000

2) Complex view Example:-

Create view of grouped records on employee table

```
Create view Emp_view AS
Select DeptID, Avg(salary)
From Employee
GroupBy DeptID;
```

O/P:- Emp_view

DeptID	Avg(salary)
1	3000.00
2	4000.00
3	4250.00

Features	Simple views	Complex views
No. of tables	one	one or more
Containing functions	NO	Yes
Contain Group of data	NO	Yes
DML through view	Yes	Not allowed
Features	Simple views	Complex views

3) Inline view:- An inline view is a select statement in the from-clause of another select statement to create a temporary table that could be referenced by the select statement. Inline views are utilized for writing complex SQL queries without Join and Subqueries operations. This is called a temporary table because a duplicate copy of the data returned by the stored subquery wasn't stored in the database.

```
Select column_names, ....
from (subquery)
where Rownum <= N;
```

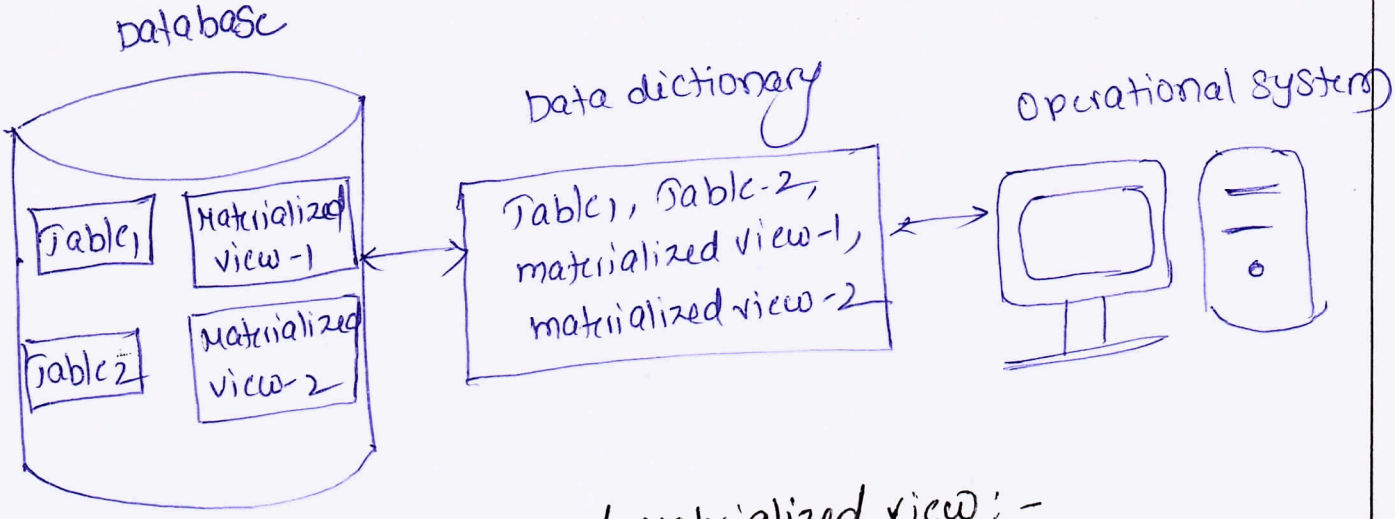
4) Materialized view:- Materialized view replicates the retrieved data physically. This replicated data can be reused without executing the view again. This type of view is also known as "SNAPSHOTS". Materialized view reduce the processing time to regenerate the whole data. It helps remote users to replicate data locally and improve query performance. The challenging part here is to synchronize the changes in materialized views underlying tables.

Creating Materialized view by filtering records using where clause

```
create materialized view emp_view
AS
select EmployeeID, Ename
from employee
where DeptID = 2;
```

O/P:- emp_view

EmployeeID	Ename	DeptID	salary
1001	John	2	4000
1004	David	2	5000
1005	Mark	2	3000



* Comparison b/w view and Materialized view :-

View	Materialized view
View is a logical structure of the table which will be used to retrieve data from one or more table	Materialized views are also logical structure but data is physically stored in database
Data access is slower compared to materialized views	Data access is faster compared to simpler view because data is directly accessed from physical location
views are generally used to restrict data from database	Materialized views are used in data warehousing

* Transaction control Commands

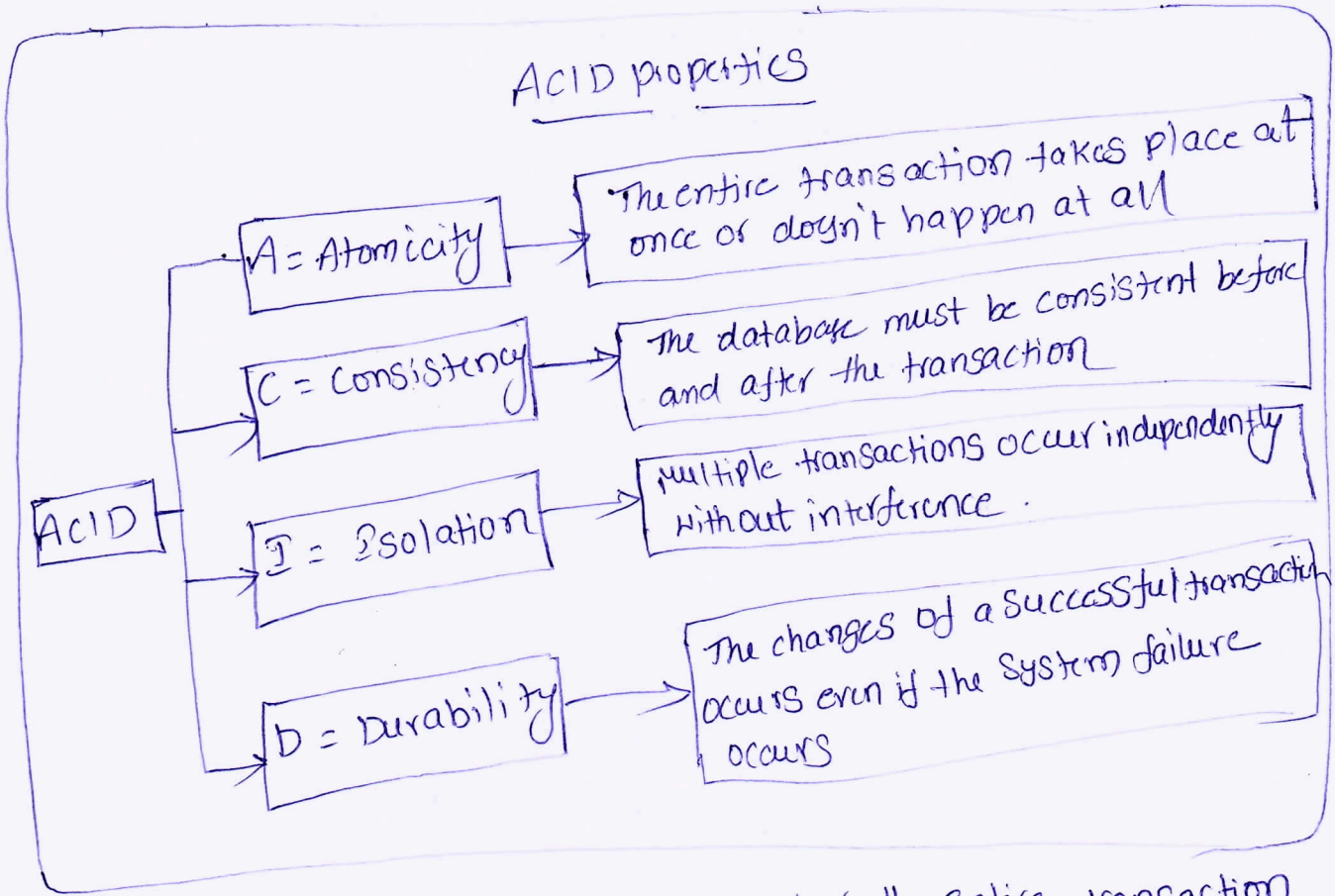
What is Transaction ?:.

A transaction is an action or series of actions that are being performed by a single user or application program which reads or updates the contents of the database.

A transaction can be defined as a logical unit of work on the database. This may be an entire program, a piece of a program, or a single command (like the SQL commands such as insert or update), and it may engage in any number of operations on the database. In the database context, the execution of an application program can be thought of as one or more transactions with non-database processing taking place in between.

ACID properties:-

A transaction is a single logical unit of work which accesses and possibly modifies the contents of a database. Transactions access data using read and write operations. In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called ACID properties.



Atomicity:- By this, we mean that either the entire transaction takes place at once or doesn't happen at all. There is no midway. i.e; transactions do not occur partially. Each transaction is considered as one unit and either runs to completion or is not executed at all. It involves the following two operations.

- **Abort:-** If a transaction aborts, changes made to database are not visible.
- **Commit:-** If a transaction commits, changes made are visible.

Atomicity is also known as the 'All or nothing rule'.

Ex:- Consider the following transaction T consisting of T_1 and T_2: Transfer of 100 from account x to account y.

Before: X: 500	Y: 200
Transaction T	
T_1	T_2
Read(X)	Read(Y)
X := X - 100	Y := Y + 100
Write(X)	Write(Y)
After: X: 400	Y: 300

If the transaction fails after completion of T_1 but before completion of T_2. (say, after write(x) but before write(y)), then amount has been deducted from x but not added to y. This results in an inconsistent database state. Therefore, the transaction must be executed in entirely in order to ensure correctness of database state.

Consistency:- This means that integrity constraints must be maintained so that the database is consistent before and after the transaction. It refers to the correctness of a database.

Ex:- The above example:
The total amount before and after the transaction must be maintained.

Total before T occurs = 500 + 200 = 700
Total after T occurs = 400 + 300 = 700

Therefore, database is consistent. Inconsistency occurs in case 2. Completes but T_2 fails. As a result T is incomplete

Isolation:- This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of database state. Transactions occur independently without interference. Changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction is written to memory or has been committed. This property ensures that the execution of transactions concurrently will result in a state that is equivalent to a state achieved these were executed serially in some order.

Ex:- Let X=500, Y=500

Consider two transactions T and T''.

T	T''
Read (X)	Read (X)
X:=X*100	Read (Y)
Write (X)	Z:=X+Y
Read (Y)	Write (Z)
Y:=Y-50	
write	

Suppose T has been executed till Read (Y) and then T'' starts. As a result, interleaving of operations takes place due to which T'' ready correct value of X but incorrect value of Y and sum computed by

T'': (X+Y=50,000+500=50,500)

is thus not consistent with the sum at end of transaction.

T: (X+Y=50,000+450 = 50,450)

This results in database inconsistency, due to a loss of 500nits. Hence, transaction must take place in isolation and changes should be visible only after they have been made to the main memory.

Durability:- This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs - These updates now become permanent and are stored in nonvolatile memory The effects of the transaction, thus, are never lost-

The ACID database properties define SQL database by modification when saved.

properties to ensure consistent, safe and robust database

The ACID properties, in totality, provide a mechanism to ensure correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations and updates that it makes are durably stored -